

UnDiffused

Technical Audit and Strategic Direction

An assessment of the existing AI-generated-image detection extension, a review of the current research and competitive landscape, and the rebuild plan. Also serves as the interview-preparation record for this project.

Field	Value
Repository	UnDiffused-AI
Branches audited	main (8bb952b), and claude/undiffused-v2-plan-gwCc0 (e2bb02f)
Scope	Inference correctness, model provenance, research review, market position, rebuild direction
Status of findings	All technical claims independently verified against source and model binaries
Target	Applied machine-learning / MLE roles; the full pipeline from data to measured on-device latency
Framing	On-device compression, not detection accuracy. See section 9.1.
Decisions taken	Calibrated detection with an abstention band; distillation to a deployable student; full architectural restructure; rigorous artifact first, store listing second; version one absorbed into the rebuild rather than presented separately

This document contains no schedule or time estimates. Stages are presented in dependency order; each stage assumes the previous one is complete.

Contents

- 1** Executive summary
- 2** What the project is today
- 3** Audit finding one: the inference path is numerically broken
- 4** Audit finding two: the models have no provenance
- 5** Audit finding three: the premise is harder than assumed
- 6** Assessment of the V2 branch
- 7** What the research literature establishes
- 8** Market position and the Google problem
- 9** Verdict: the problem worth solving
- 10** Proposed direction: a three-layer architecture
- 11** The rebuild, in dependency order
- 12** Compute and data constraints
- 13** Risks and open questions
- 14** Positioning and framing
- A** Evidence log
- B** References

1 Executive summary

UnDiffused is a Chrome extension that claims to detect AI-generated images locally, on-device, using a dual-model ensemble. The privacy architecture is real and works. The detection does not.

The project's poor accuracy has been attributed to the model being weak. That diagnosis is incomplete. There are three separate failures, and the model's quality is the least important of them.

First, the inference code is arithmetically incorrect. The global model emits three class logits per image. The code handles only the one-class and two-class cases, so every global prediction falls through to a branch that reads the wrong array positions and misinterprets raw logits as probabilities. The resulting number is not a weak signal; it is noise. That noise then drives an early-exit gate which, when triggered, skips the second model entirely and reports the noise as the final verdict.

Second, the shipped models cannot be described. Two 87 MB ViT-Base/16 checkpoints are bundled with no record of their training data, no label map, no provenance and no measured accuracy. The meaning of the global model's three classes is unknown. Which output index corresponds to "AI" was never established; the source code preserves the original author's unresolved deliberation on the question as a comment, and the guess it settled on may be inverted.

Third, the product promise is not achievable as stated. Reliable universal detection of AI-generated images is an open research problem. The strongest published detectors lose more than twenty points of accuracy when moved from curated benchmarks to real-world images.

The recommendation is not to abandon the project but to change what it claims. The proposal is to move from a verdict machine to an evidence machine: a local-first tool that establishes provenance deterministically where provenance exists, offers a calibrated and openly uncertain model opinion where it does not, and exposes forensic evidence in both cases. This is the only version of the project that can be made to work honestly, and it occupies precisely the gap that the detection feature Google has built into Chrome cannot cover.

Alongside that, the question the project is measured against changes. Judged on detection accuracy it competes with cloud services whose advertised figures are unattached to any stated benchmark, and an honest number loses to a dishonest one. Judged on whether a model of competitive quality can be compressed to run inside a browser at usable latency without surrendering its accuracy, the result is defensible, the measurements are the deliverable, and the cloud services offer no comparison because they do not attempt the problem. Section 9.1 sets this out. It is a change of question, not of marketing.

A secondary conclusion concerns what is valuable in the existing repository. The ten from-scratch forensic analysis tools — among them a radix-2 Cooley-Tukey 2D FFT, a complete Canny edge detector with non-maximum suppression and hysteresis, JPEG quantization-table estimation and PRNU residual analysis, all implemented in TypeScript and running in a browser — are the hardest part of this codebase to reproduce, and they are correct. They are currently presented as a secondary feature beneath a broken classifier. They should become the substance of the product.

2 What the project is today

UnDiffused is a Manifest V3 Chrome extension built with React, TypeScript and Vite, using the CRXJS plugin. Inference runs through ONNX Runtime Web on a WebAssembly backend. Roughly 8,000 lines of TypeScript sit under `src/`.

2.1 Runtime components

Component	File	Responsibility
Service worker	<code>src/background/main.ts</code>	Registers the right-click context menu; fetches image bytes on the extension's behalf to bypass page CORS restrictions; relays scan triggers to the active tab.
Content script	<code>src/content/index.tsx</code>	Injected into every page. Mounts the React interface inside a Shadow DOM so host-page CSS cannot affect it.
Popup	<code>src/popup/Popup.tsx</code>	Drag-and-drop or file-picker entry point; converts the file to a data URL and dispatches a scan.
Inference worker	<code>src/content/inference/worker.ts</code>	Inline Web Worker. Loads both ONNX sessions, builds tensors, runs inference, fuses scores.
Offscreen document	<code>src/offscreen/</code>	Bundled by the build but never instantiated. Dead code.

2.2 The scan path as currently implemented

A right-click triggers a message to the content script, which resolves the highest-resolution candidate from the image's `srcset`, fetches the bytes through the service worker, and decodes them to an `ImageBitmap`. That bitmap is transferred to the worker together with a set of crop rectangles.

The worker then runs a two-stage process. It first cuts the image into four quadrants, resizes each to 224x224, and runs them through the global model, averaging the four results. If that average falls outside the band 0.05 to 0.95, the scan stops and reports the global score. Otherwise the local model runs over saliency-ranked 224x224 crops, the three highest local scores are averaged, and the two stages are combined as a fixed weighted blend of 25 per cent global and 75 per cent local. A threshold of 0.5 decides the verdict.

2.3 The forensic toolkit

Ten independent analysis tools are implemented under `src/components/tools/`. Each computes its result in-browser, from scratch, with no external library.

Tool	What it computes
ELA	Re-encodes the image at a chosen JPEG quality, takes the per-pixel absolute difference, normalises to the 95th percentile and applies a gamma curve.
FFT	Reduces to 256x256 greyscale, runs a recursive radix-2 Cooley-Tukey 2D FFT, and reports radial band energies plus periodic grid-peak detection.
PRNU	Estimates a sensor-noise residual by box-blur denoising, then measures block-wise variance consistency and Sobel edge correlation.
Noise	Laplacian high-pass residual on luminance, per-block variance, and a uniformity measure from the coefficient of variation.
Compression	Per-block 8x8 DCT scored against candidate quantization tables to estimate the JPEG quality factor and count inconsistent blocks.

Tool	What it computes
Clone	Mean-pooled normalised block descriptors, FNV-1a hash bucketing, and cosine similarity to find duplicated regions.
Gradient	Sobel and Laplacian edge maps plus a full Canny implementation with Gaussian smoothing, non-maximum suppression and hysteresis.
Highlight	Locates specular highlights above a brightness threshold and estimates a gradient-weighted light direction for each.
Aberration	Measures per-channel edge-angle separation at Sobel edges to detect chromatic aberration.
Metadata	Hand-written JPEG segment parser covering APP1 (EXIF, XMP) and APP13 (IPTC), with keyword matching on generator software tags.

Two further files in that directory, `ToolActionRow` and `ToolSlider`, are shared user-interface components rather than analyses.

3 Audit finding one: the inference path is numerically broken

This is not a tuning problem or a model-quality problem. The scores the extension produces are arithmetically wrong, and would remain wrong with a perfect model behind them.

3.1 Ground truth from the model binaries

Both ONNX files were opened directly and their graphs inspected. The results are unambiguous.

Property	model_global_quantized.onnx	model_local_quantized.onnx
Input tensor	pixel_values	pixel_values
Input shape	[batch_size, num_channels, height, width]	[batch_size, num_channels, height, width]
Output tensor	logits	logits
Output shape	[batch_size, 3]	[batch_size, 2]
classifier.weight	[768, 3]	[768, 2]
Patch embedding	[768, 3, 16, 16]	[768, 3, 16, 16]
Encoder layers	12	12
Implied architecture	ViT-Base/16	ViT-Base/16
File size (bytes)	87,346,871	87,333,629
Quantization	dynamic int8 (onnx.quantize 0.1.0)	dynamic int8 (onnx.quantize 0.1.0)
Embedded label map	none	none

Two consequences follow immediately. The global model is a **three-class** classifier, not a binary one. And the README's description of a "dual-model ensemble" combining architecturally distinct global and local models is inaccurate: both files are the same architecture at essentially the same size, differing only in the width of the final classification head.

3.2 The defect

The output-parsing routine in `src/content/inference/worker.ts` reads:

```
const classCount = dims[1] || 1;

if (classCount === 2) {
  const logits = Array.from(outputData.slice(i * 2, (i + 1) * 2));
  const probs = softmax(logits);
  probabilities.push(probs[0]);
} else {
  const raw = outputData[i];
  const val = raw >= 0 && raw <= 1 ? raw : sigmoid(raw);
  probabilities.push(val);
}
```

For the global model `classCount` is 3, so the two-class branch is never taken and control passes to the `else`. Two independent errors occur there.

- **Wrong array positions.** The branch reads `outputData[i]`. With three classes per image, image *i*'s logits occupy positions `3i`, `3i+1` and `3i+2`. Scanning the four quadrants therefore reads flat positions 0, 1, 2 and 3 — the first quadrant's three class logits followed by the second quadrant's first logit. Values are mixed across both class and batch boundaries.

- **Wrong activation.** A raw logit is accepted unchanged as a probability whenever it happens to land between 0 and 1, and squashed through a sigmoid otherwise. Neither is a valid way to recover a class probability from a three-way softmax head.

The global score is therefore noise. This matters more than it might appear, because the same value controls the early-exit gate:

```
const isUncertain = globalAiProb > 0.05 && globalAiProb < 0.95;
if (mode === 'deep' || isUncertain) { /* run the local model */ }
```

When the noise happens to fall outside that band, the local model — the only component whose output is parsed correctly — is skipped entirely, and the noise is reported to the user as the final verdict.

3.3 The unresolved class mapping

The local model does have two classes, so it takes the softmax branch. But which index means "AI" was never determined. The source retains the original deliberation verbatim:

```
probabilities.push(probs[0]); // Class 0 is usually AI? Wait, usually Class 1 is target.
// Convention check: Usually Index 0 = Real, Index 1 = Fake/AI?
// Or Index 0 = AI, Index 1 = Real?
// In standard "UniversalFakeDetect", 0=Real, 1=Fake.
// Let's assume standard: 1 = AI.
// WAIT - In previous worker code: `aiProb = probs[0]`.
// I will stick to the previous code's convention: Index 0 is AI.
```

This is an unresolved coin-flip that was shipped. There is a live possibility that the detector has been **inverted** throughout its existence, labelling authentic photographs as generated and generated images as authentic. Establishing which is the case is the first task of the rebuild, and the answer forms the opening result of the project's benchmark record.

Why this class of bug survives. Ordinary application code fails loudly — an exception, a failed request, a blank screen. Machine-learning inference code fails silently: incorrect indices still yield finite numbers, a sigmoid still returns something in the interval from 0 to 1, and a user interface will render "87% AI" from meaningless input without complaint. The only defences are asserting the tensor contract at load time and testing against a small set of images whose true labels are known. Neither existed in this codebase.

4 Audit finding two: the models have no provenance

Beyond the parsing defect, the two checkpoints cannot be described in any of the ways that would be required to defend them.

Question	Answer available in the repository
What data were they trained on?	Unrecorded.
What do the global model's three classes represent?	Unrecorded. No label map is embedded in the ONNX metadata, and none is shipped alongside.
Which output index corresponds to AI?	Never established. Assumed, with the assumption disputed in a code comment.
Who trained them, when, and under what licence?	Unrecorded.
What accuracy do they achieve, on what dataset?	Never measured.
Do the training scripts in the repository correspond to them?	No. The V2 branch's <code>train_global.py</code> constructs a ResNet-50; the shipped checkpoints are Vision Transformers.

The git history compounds this. Commit 78c5ecd is titled "NEW Model - Quantised Swin Tiny ONNX model". The checkpoints currently in the repository are ViT-Base/16, a different architecture family with roughly four times the parameter count of Swin-Tiny. The commit message does not describe the artefact.

For a portfolio project this is the most exposed surface in the repository. Shipping 174 MB of weights that cannot be described is among the first things a competent reviewer will probe, and there is no defensible answer. The checkpoints are also not salvageable by tuning, because there is nothing to tune towards: without knowing the class semantics, no threshold or fusion weight can be chosen on principle.

Recommendation. Both files should be removed rather than repaired. Their replacement is described in section 10.

5 Audit finding three: the premise is harder than assumed

Suppose the parsing were fixed and a properly trained, fully documented model installed. The product would still not deliver what its name promises, because reliable universal detection of AI-generated images is an open research problem. Section 7 sets out the evidence; the summary is that detectors fail in three characteristic ways.

- **Generator overfitting.** A detector trained on one family of generators learns that family's fingerprint rather than a general notion of synthetic origin, and degrades sharply on generators it has not seen. This is the most consistently replicated result in the field.
- **Laundering.** The artefacts detectors rely on are predominantly high-frequency. Resizing, recompression and screenshotting attenuate or destroy them. Ordinary distribution through social platforms and messaging applications performs exactly these operations.
- **Benchmark inflation.** Standard evaluation sets are substantially easier than real circulating imagery, so published accuracy figures systematically overstate deployed performance.

6 Assessment of the V2 branch

The branch `claude/undiffused-v2-plan-gwCc0` (head commit `e2bb02f`, substantive commit `e88420d`) adds 1,831 lines across 24 source files, excluding build output. It warrants a split verdict.

6.1 What it gets right

The branch correctly diagnoses most of finding one. It introduces a metadata contract loader, a separate fusion module, overlapping deep-scan tiling, lazy loading of the second model, 34 unit tests, Python training scripts and a benchmark protocol document. The architectural direction is sound and much of this scaffolding is worth carrying forward.

6.2 What it gets wrong

It replaces guesses with differently-worded guesses, and gives them the appearance of rigour.

The metadata does not match the models. The generated `model_global_meta.json` declares:

```
{ "input_name": "input", "output_name": "output",
  "num_classes": 3, "ai_class_index": 1, ... }
```

The actual tensors are named `pixel_values` and `logits`. A file whose stated purpose is to remove hardcoded assumptions about the model states the model's interface incorrectly, and the accompanying strict load-time validator would reject the very checkpoints the branch ships with. The value `ai_class_index: 1` is a fresh guess, inverted from the previous guess of 0, with no evidence offered for either.

The learned fusion head was never learned. `fusion_v2.json` contains:

```
{ "type": "logistic_regression",
  "weights": [0.30, 0.50, 0.40, 0.30, 0.20, 0.50],
  "bias": -0.10, "temperature": 1.0, "threshold": 0.50 }
```

These are hand-chosen round numbers. The defining property of a logistic regression is that its coefficients are fitted to data; these were not. A temperature of exactly 1.0 means the advertised temperature scaling is a no-op, so the calibration step the branch claims to add does nothing at all.

No new weights were produced. A diff restricted to `public/models/*.onnx` between main and `e2bb02f` is empty. The same two undocumented checkpoints remain.

The training scripts do not match the artefacts. `train_global.py` builds a ResNet-50 with an ImageNet-pretrained backbone. The shipped models are ViT-Base/16. Running the pipeline as written would produce something incompatible with the metadata the same branch generated.

Verdict. Keep the structure; discard every number. The metadata-loader pattern, the test harness, the overlapping-tile logic and lazy model loading are genuinely useful. Every constant in the JSON files is fictional, and the 34 tests validate the fiction rather than the models. This is scaffolding for a fix that was mistaken for the fix itself.

7 What the research literature establishes

7.1 Why detection is possible in principle

Cameras and generative models produce measurably different image statistics. A camera image carries a sensor noise fingerprint, a demosaicing pattern from the colour filter array, and a particular compression history. A diffusion model constructs an image by iterative denoising, which leaves characteristic traces in the frequency domain and in local texture correlations. A classifier can learn to separate these.

7.2 Why it fails in practice

Finding	Evidence
Detectors do not transfer across generators.	The most consistently reproduced result in the field. Training on a narrow set of generators yields a detector that approaches chance on unseen families.
Real-world transformation destroys the signal.	The NTIRE 2026 challenge was built around this: 108,750 real and 185,750 generated images from 42 generators, augmented with 36 transformations. 511 teams registered and 20 submitted. The organisers' stated conclusion is that the problem is not yet solved.
Standard benchmarks are too easy.	The Chameleon benchmark collects roughly 26,000 generated images that passed a human perception test — annotators judged them real. Detectors scoring above 90 per cent elsewhere collapse on it; MiraGe falls to 69.06 per cent.

7.3 What demonstrably helps

The same literature is unusually clear about which interventions work, and they are cheap.

Intervention	Reported effect
A linear classifier on a frozen modern vision foundation model, in place of a bespoke detector architecture.	Gains exceeding 20 percentage points of accuracy on in-the-wild benchmarks. The authors' framing is that the raw capability of an updated foundation model outweighs the craftsmanship of a static purpose-built detector.
Choosing a self-supervised backbone over a language-aligned one.	DINOv2-L/14 reaches 94.90 per cent average AUC, ahead of CLIP variants. The proposed explanation is that image-text alignment introduces semantic shortcuts which bypass the fine-grained cues detection depends on.
Increasing the diversity of generators in the training set.	One method improves from 80.83 to 94.90 per cent average AUC when trained on diverse in-the-wild data rather than a single laboratory corpus. Community Forensics reports monotonic improvement as generator count rises, using 2.76 million images from 4,803 models.
Texture-based crop selection instead of centre cropping.	94.90 versus 91.26 per cent average AUC for the same method.
Post-hoc calibration of the decision threshold.	Detectors are systematically miscalibrated and tend to misclassify generated images as real. A learnable scalar correction applied to the logits, fitted on a small validation set with the backbone frozen, recovers much of the loss without retraining.
A frozen DINOv3 backbone with a lightweight probe over selected tokens.	Generalises because it leans on global low-frequency structure rather than generator-specific high-frequency artefacts — precisely the cues that survive compression and resizing.

The methodological shift. The field has inverted since roughly 2023. The older recipe was to engineer clever handcrafted artefact features. The current recipe is to take a large frozen self-supervised backbone and train a linear layer on top of it. This trains on the order of a thousand parameters instead of tens of millions, and it generalises better precisely because a frozen backbone cannot overfit to the generators in the training set.

8 Market position and the Google problem

8.1 Google now ships detection inside Chrome

At Google I/O 2026, Google announced AI-image detection integrated directly into Chrome, Google Search and Circle to Search. On desktop Chrome this appears as a right-click option on any image — the same interaction UnDiffused uses.

The limitation is decisive, and it is the project's opportunity. Google's approach is **provenance-based, not detection-based**. It reads SynthID watermarks and C2PA Content Credentials: cryptographic signatures embedded at the moment of generation. It works for participating generators, which include OpenAI, Nvidia, Google's own models, ElevenLabs and Kakao. It cannot flag output from Midjourney, from Stable Diffusion, from any locally-run model, or from any image whose metadata has been removed.

Metadata removal is not an edge case. Messaging applications strip it, email does not preserve it, most content management systems do not carry it, and taking a screenshot defeats it completely. C2PA adoption is real and growing — over 6,000 members and affiliates, with capture-time signing shipping in cameras from Leica, Sony, Nikon, Canon and Samsung — but the chain breaks the first time an image passes through a tool that does not understand it.

8.2 Where the coverage gaps are

Image category	Chrome built-in	Cloud detectors	Status
Watermarked or C2PA-signed	Covered	Covered	Solved; not a differentiator
Midjourney, Stable Diffusion, local models	Blind	Cloud-only, unverifiable accuracy claims	Open
Screenshotted or heavily recompressed	Blind	Degrades substantially	Open
Private or sensitive images	Not applicable	Requires uploading the image to a server	Open

8.3 The competitive field

Every significant competitor — Hive AI Detector, AI or Not, BitMind, Copyleaks, Is It AI — is a cloud service. Each uploads the image to a server for analysis. Each advertises accuracy at or above 95 per cent without naming the benchmark the figure refers to.

Independent testing does not support those claims uniformly. Hive scored 98.03 per cent in a 2024 University of Chicago study, but 88 per cent with a 9 per cent false-positive rate in an independent March 2026 benchmark. The gap between marketed and measured accuracy is the norm in this category rather than an exception.

The strategic consequence. Honesty is available as a differentiator because no competitor has taken it. A tool that publishes its benchmark, states its false-positive rate, declares an abstention band and documents its evaluation protocol would be the only one in the category able to defend its numbers under scrutiny. That property is also, and not coincidentally, what makes the work legible as engineering rather than as a demonstration.

9 Verdict: the problem worth solving

9.1 Restating the problem

The framing inherited from version one was "detect AI-generated images." Judged on that question, the project competes directly with well-funded cloud services on raw accuracy, and it will lose — not because of any defect in the build, but because the honest number is lower than the numbers competitors advertise without evidence. Optimising a portfolio project against a metric where the honest answer looks worse than the dishonest one is a losing position by construction.

There is a different question underneath the same system, and it is the one actually worth answering:

Can a forensics model of competitive quality be compressed to run entirely inside a browser, at usable latency, without surrendering the accuracy that made it worth deploying?

This is not a rhetorical repositioning. It changes which numbers are the result. Under the detection framing the headline is accuracy, and every competitor claims a higher one. Under the compression framing the headline is the **compression ratio, the on-device latency, and the proportion of the teacher's accuracy the student retains** — and on those, the cloud services offer no comparison at all, because none of them attempt on-device execution. They are not competitors who score better; they are addressing a different problem.

Accuracy does not disappear under this framing. It becomes a supporting measurement in a table, reported honestly alongside the benchmark it was obtained on, rather than the single number the project is judged by.

	Detection framing	Compression framing
Headline metric	Accuracy on a benchmark	Compression ratio, latency, accuracy retention
Position against cloud services	Direct competitor, scoring worse	Not comparable; they do not attempt on-device execution
Effect of an honest number	Undercuts the claim	Strengthens it — retention is only meaningful if measured
What the engineering demonstrates	Model training	Distillation, quantization, calibration, and browser-runtime deployment

9.2 Not achievable

Reliable universal detection — answering "is this image AI-generated?" correctly for arbitrary images from arbitrary generators in arbitrary states of compression — cannot be delivered. Not by this project, and not currently by anyone. Building toward that promise guarantees a product that is confidently wrong often enough for users to notice and say so.

9.3 Achievable

- A detector that generalises **measurably better than the naive baseline** on a stated, reproducible benchmark, with generator families held out of training.
- A system that is **honestly calibrated** — one that reports uncertainty when uncertain rather than producing a confident number regardless.
- The only **local and private** option in a category composed entirely of cloud services.
- Coverage of exactly the images that **Chrome's built-in feature cannot see**.
- A **measured compression result**: a large teacher distilled into a student small enough to run in a browser, with the accuracy cost of that reduction quantified rather than assumed.
- A **published evaluation protocol and benchmark record**, including unfavourable results.

9.4 The trade being accepted

An honest headline number will trail competitors' marketing claims. A realistic expectation is in the region of 85 to 92 per cent on held-out generator families, degrading on the hardest circulating imagery. Competitors will continue to advertise figures above 95 per cent. This project will look worse on a landing page and better under any examination. That is the trade, and it has been accepted deliberately.

10 Proposed direction: a three-layer architecture

The reframing is from a **verdict machine** to an **evidence machine**: a local-first tool that answers "what can actually be established about this image?" through three layers, only one of which is probabilistic — and that one is permitted to abstain.

Layer one: provenance

Deterministic and always correct. Parse C2PA manifests, EXIF and XMP. Where a valid signature exists, report a certain answer rather than an estimate. Much of the groundwork already exists in the Metadata tool, which parses JPEG APP1 and APP13 segments by hand and already matches generator software tags. Extending it to full C2PA manifest verification adds no machine-learning risk and covers the signed case at least as well as any competitor.

Layer two: a calibrated model that abstains

A frozen self-supervised vision backbone with a trained lightweight head, temperature-scaled, reporting three states rather than a percentage: likely authentic, inconclusive, likely AI-generated. The inconclusive band is tuned to a target false-positive rate, on the reasoning that wrongly accusing a genuine photograph is the more damaging error.

Layer three: forensic evidence

The ten existing tools, re-framed. Presented as "here is the measurable evidence, and here is the model's opinion alongside it", they stop being an accessory and become the product's substance. No competitor offers anything comparable, and their per-tool output is honest by construction because each reports a measurement rather than a judgement.

There is a tension here that must be designed around rather than ignored. The forensic layer is the project's most distinctive engineering, but it is also a power-user surface, and presenting ten analyses to someone who asked a yes-or-no question is poor product design. The resolution is to default to the simple three-state answer alongside any provenance result, and to place the forensic tools behind a single "examine the evidence" affordance. Simple by default, deep on demand.

10.1 Backbone selection and the licensing constraint

The backbone family should be **DINOv2**, and that part of the decision is as much legal as technical. The *size* within that family is a measured decision rather than an assertion, and is addressed below.

Candidate	Licence	Assessment
DINOv2	Apache 2.0	Recommended. Permissive; weights may be redistributed inside a Chrome Web Store package without further conditions. Strong reported performance on this task.
DINOv3	Custom Meta licence	Stronger on benchmarks, but redistribution carries licence-propagation conditions and access is gated behind an approval process. Shipping weights inside a published extension is redistribution.
CLIP variants	Varies	Reported to underperform self-supervised backbones on this task, attributed to semantic shortcuts arising from image-text alignment.

Because the extension must ship the weights inside its package, redistribution terms are a hard constraint rather than a preference. This is a real engineering decision with a defensible justification, and it is worth stating explicitly in the project's documentation.

10.2 Capacity, and why distillation is required rather than optional

The published results that motivate this approach were obtained with **large** backbones: the 94.90 per cent average AUC figure cited in section 7 belongs to DINOv2-L/14. A small backbone will not match it, and assuming otherwise would repeat exactly the error this document criticises elsewhere. Backbone size must therefore be evaluated, not asserted.

Backbone	Approximate int8 size	Role
ViT-L/14	roughly 300 MB	Far too large to ship. Viable only as an offline teacher.
ViT-B/14	roughly 86 MB	Shippable but heavy. A fallback if the student underperforms.
ViT-S/14	roughly 22 MB	The deployment target. Reaches useful quality only if distilled.

This is what makes distillation a requirement rather than an embellishment. The backbone that performs best cannot be shipped in a browser extension, so compressing its behaviour into something that can is not an academic exercise — it is the only route by which the product exists at all. A large frozen teacher produces outputs once; those outputs are cached; a small student is trained to reproduce them. Because the teacher's forward passes are computed a single time, the whole procedure remains within reach of modest hardware.

The size consequence is substantial either way. An int8-quantized ViT-S/14 is approximately 22 MB against the 174 MB currently shipped — roughly an eightfold reduction — and the distillation step is what allows that reduction to come without surrendering the accuracy the larger backbone provides.

11 The rebuild, in dependency order

The stages below are ordered by dependency. Each assumes its predecessors are complete. No duration is implied.

11.1 Where value lands

The stages are not uniform in weight, and the plan is deliberately arranged so that usable results arrive before the largest block of work begins rather than only at the end.

Stages	Character	What becomes true on completion
1 and 2	Small, self-contained, no dependency on new data or training	The current system's real behaviour is known, including whether it was inverted, and a reproducible measurement harness exists. The project can be described accurately from this point on.
3 to 6	The substantial block: data, feature extraction, baseline, distillation	The central result exists — a measured teacher, baseline and student, and the compression and retention figures that section 9.1 identifies as the headline.
7 to 9	Engineering and packaging	The result is deployed, verified end to end on real hardware, and published with its evaluation protocol attached.

The practical consequence is that stages 1 and 2 are worth completing on their own even if the larger block is deferred. They convert every description of the project from assertion into measurement, which is the single change that most improves how the work can be discussed.

Stage 1 — Establish ground truth on the existing system

Before changing anything, answer the question the repository never answered: what do the current models actually do? Assemble a small labelled set — on the order of fifty known-authentic photographs and fifty known-generated images — and run both checkpoints against it directly, outside the extension.

This determines conclusively whether the shipped detector was inverted, and produces the honest "before" measurement that anchors every subsequent comparison. It is the opening entry in the project's benchmark record and the first paragraph of its written account.

Stage 2 — Build the measurement harness before the model

Construct the evaluation apparatus first. This ordering is deliberate: the V2 branch's central failure was writing tests that validated invented constants, which is what happens when the harness follows the implementation instead of preceding it.

- Generator-family holdout splits, so that cross-generator generalisation is measured rather than assumed.
- Laundering augmentations applied at evaluation time: JPEG quality sweeps, resize chains, screenshot re-capture.
- Metrics that reflect deployment: AUROC, false-positive rate at 95 per cent true-positive rate, and expected calibration error.
- A fixed, versioned, reproducible protocol, so that later numbers remain comparable to earlier ones.

Stage 3 — Resolve licensing, then assemble data by streaming rather than downloading

Settle the licensing question before any data is collected, not after. Community Forensics is released under CC-BY-4.0, but its dataset card describes it as intended for research purposes and the individual generator models within it largely carry CreativeML OpenRAIL-M terms. Whether a model trained on it may be distributed inside a public extension listing is a question with a real answer, and it must be obtained now, because it can

constrain both the data sources and how the finished product is framed.

Generator diversity is the highest-leverage variable identified in the literature, which points to Community Forensics as the primary source. The full corpus is 2.76 TB and the reduced variant remains in the hundreds of gigabytes; neither can reasonably be stored locally.

The resolution is to stream the dataset, extract features on the fly, and never write an image to disk. Real photographs are drawn from the licensed real-image data paired with the same corpus and from open photographic datasets. Near-duplicates are removed across splits, and splits are cut by generator family.

Stage 4 — Extract features once with a frozen backbone

Run each training image through the frozen DINOv2 backbone a single time and retain only the resulting embedding vector. This is the sole compute-intensive step, and it is a forward pass rather than a training run.

Extract at more than one backbone size — small, base and large — so that the capacity question posed in section 10.2 is answered by measurement. The large variant's outputs additionally serve as the teacher signal in stage 6.

Caching embeddings converts every subsequent modelling experiment into a cheap operation on a small matrix. This single decision is what allows the entire project to proceed on modest hardware.

Stage 5 — Train a baseline head, calibrate, and set the abstention band

Fit a lightweight head — logistic regression or a small multilayer perceptron — on the cached embeddings. This is the baseline against which every later result is compared, and it should be recorded as such rather than presented as the finished model.

Apply temperature scaling on a held-out validation split so that reported confidences correspond to observed frequencies. Then choose the two thresholds bounding the inconclusive band, targeting a stated false-positive rate rather than maximising raw accuracy.

- **Acceptance criterion:** the abstention rate is bounded, not free. A model that returns "inconclusive" on a large share of ordinary images is not honest but useless, and the positioning inverts from candour to evasion. Fix a ceiling in advance — on the order of one image in four — and select thresholds subject to it. If no useful false-positive rate is reachable within that ceiling, the design has failed and that must surface here rather than at launch.

Stage 6 — Distil a large teacher into a deployable student

The backbone that performs best cannot be shipped. Use the largest evaluated backbone as an offline teacher, cache its outputs across the training corpus, and train a small student to reproduce them. Because the teacher runs only once, the cost of this stage is dominated by student training on cached targets, which is tractable on the available hardware.

This is the stage where the substantive modelling decisions live: what to match (logits, intermediate features, or both), the distillation temperature, how the distillation loss is weighted against the ground-truth objective, and how much student capacity is actually required. Report the student against both the teacher and the stage 5 baseline; the gap between the three is the project's central result.

Stage 7 — Export, quantize, and verify the deployed artefact

Export to ONNX and quantize to int8, then verify that the quantized model's outputs match the unquantized reference within tolerance on the evaluation set. Quantization is a lossy transformation and must be measured, not assumed.

Ship a metadata file stating the input and output tensor names, the class count, the class-to-label mapping and the normalisation constants — with those values read from the exported model rather than written by hand.

Assert the contract at load time and fail loudly on mismatch.

Stage 8 — Restructure the extension around the three layers

Rebuild the runtime around provenance, then calibrated model, then forensic evidence. Extend the existing metadata parser to full C2PA manifest verification. Reorganise the interface to present evidence rather than a single verdict.

Add a WebGPU execution path with WebAssembly fallback. Narrow the permissions: the extension currently requests activeTab without using it and injects a content script into every page unconditionally, both of which invite review scrutiny and neither of which is necessary. Remove the dead offscreen document, whose worker references a model file that does not exist.

Stage 9 — Publish the artifact, then ship the listing

The order matters and is deliberate. Complete the public artifact first: the repository, the benchmark report and the written account, to a standard that stands on its own and can be installed from source. This is the part that bears the most weight under technical questioning, and it must be settled before any public claim is attached to it.

Only then prepare the store listing: privacy policy, screenshots and description. The local-first architecture is a genuine selling point and should be stated plainly. Publish the benchmark alongside the extension, including the results that are unflattering. Shipping is the packaging of a finished technical claim rather than a substitute for one.

11.2 Store-readiness items identified in the audit

Item	Current state
Privacy policy URL	Absent. Required for any extension handling user data. Straightforward here, and a selling point.
Listing screenshots	Absent. At least one is required, at 1280x800 or 640x400.
Permission scope	activeTab is requested but never used. Content scripts inject on all HTTP and HTTPS pages unconditionally.
Package size	174 MB. Within the 2 GB limit but a poor install experience; the proposed backbone reduces this by roughly eight times.
Dead code	The offscreen document is bundled but never instantiated, and its worker loads models/model_quantized.onnx, which does not exist.
Manifest basics	Name, version, description and all three icon sizes are present. No remote code is loaded.

12 Compute and data constraints

12.1 Available hardware

Resource	Specification	Role
Desktop GPU	NVIDIA GTX 1660 Ti, 6 GB VRAM, Turing, no tensor cores	Feature extraction. Sufficient, because the backbone is frozen and only forward passes are required.
Laptop	Intel Core Ultra 7 258V, Arc 140V integrated graphics, on-die NPU	Not a training machine. Valuable as a realistic low-end target for measuring extension latency on integrated graphics.
Google AI Pro subscription	Gemini, Deep Research, Jules, Antigravity, AI Studio limits, storage	Coding assistance only. Does not include Colab compute units or GPU access.

Google's published plan description for AI Pro does not list Colab, and Colab's own FAQ names no bundled subscription granting its paid tiers. Colab Pro is a separate purchase. Bundles change, so this is worth confirming directly from a signed-in Colab account; student offers may also apply.

12.2 Why the available hardware is sufficient

The frozen-backbone approach is well matched to this hardware, because it replaces training with a single inference pass per image.

Operation	Estimate on the available GPU
DINOv2 ViT-S/14 forward pass	Approximately 6.0 GFLOPs per image (patch-14 at 224x224 yields 256 tokens); on the order of 150 to 250 images per second when batched
Feature extraction over 200,000 images	Tens of minutes rather than days
On-disk feature cache, 200,000 images	About 300 MB (200,000 x 384 dimensions x 4 bytes)
Fitting the classification head	Seconds, on CPU
Re-running an experiment with a different head or calibration	Seconds, because the features are already cached

Figures are order-of-magnitude estimates for planning purposes and should be replaced by measurements once the pipeline runs.

The real constraint is data movement, not compute. Because the backbone is frozen, features are extracted once and every subsequent modelling experiment is nearly free. The binding limit is the size of the training corpus relative to available storage and bandwidth, which is why streaming rather than downloading is a structural decision rather than an optimisation.

13 Risks and open questions

Risk	Assessment and response
The honest headline number will trail competitors' claims.	Accepted deliberately. Mitigation is to publish the protocol and the false-positive rate so the figure is defensible, and to compete on transparency rather than on the size of the number.
Detection quality may remain unsatisfying on the hardest images.	Expected, and the reason the abstention band exists. The provenance and forensic layers continue to deliver value where the model declines to commit.
Google or another platform may extend built-in detection beyond watermark-based methods.	Possible. The local-and-private property and the forensic evidence layer are not addressed by a watermark-reading feature, so the differentiation is not solely a coverage gap.

Risk	Assessment and response
Dataset licensing may constrain a publicly distributed product.	Community Forensics is CC-BY-4.0, but individual generator models within it carry their own terms. Licence metadata must be reviewed before any commercial framing, and attribution carried.
A full restructure risks discarding work that is correct.	The ten forensic tools are the strongest code in the repository and must be preserved. The restructure changes their framing and the surrounding architecture, not their algorithms.
Quantization may erode the accuracy gained.	Measured explicitly in stage 7 rather than assumed. If int8 proves too lossy, alternatives include a larger backbone at higher quantization, or selective precision.
The modelling contribution may be judged too thin.	A frozen backbone with a linear probe is a handful of lines of library code, and a reviewer will notice. This is the reason distillation is a required stage rather than an optional one: the probe is the baseline, the student is the contribution, and the measured gap between them is the result.

Open questions

- What the global model's three classes were intended to represent — answerable only empirically, in stage 1, and of historical interest only once the checkpoints are replaced.
- Whether to target a specific vertical for launch positioning, which would sharpen the product narrative without altering the technical plan.
- Whether the abstention band should be user-adjustable, trading interpretability for control.

14 Positioning and framing

The technical plan and the way the work is described are separate problems, and optimising only the first is a mistake. What follows is deliberate rather than incidental.

14.1 The existing description does not match the artefact

This is the most urgent item in this document, and it is independent of the rebuild. The project's current written description names components that are not present in the repository.

Claim as written	What the repository contains
"built on CLIP ViT-L/14, FreqNet (ResNet50) and Swin V2 Tiny"	None of these three. Two ViT-Base/16 checkpoints, established from patch-embedding weights of [768, 3, 16, 16] and 12 encoder layers in each graph. No CLIP, no frequency-domain branch, no Swin.
"dual global/local INT8 ONNX ensemble"	Accurate as to INT8 and ONNX. "Ensemble" overstates it: the two checkpoints share one architecture and differ only in classification-head width.
"Reduced median inference to under 3 seconds"	Unverified. No benchmark exists in the repository; the pipeline's own request timeout is 30 seconds.
"3-4x throughput from WASM SIMD multithreading"	Unverified. No comparative measurement exists against a single-threaded or non-SIMD baseline.
"saliency-aware 224x224 cropping and 25/75 global/local fusion refined on the top-3 local scores"	Accurate. This matches the implementation exactly.
"Shipped a Chrome extension"	Overstated. The build output is gitignored and the extension is not published to the Chrome Web Store. It is installable from source.

The risk is asymmetric and immediate. A named component that does not exist cannot be discussed when asked about, and an interviewer who probes it will read the gap as fabrication rather than as an error. Unverified performance figures carry the same exposure: they are precisely the claims a technical interviewer selects, because they are the ones that sound measurable. Correcting the description costs very little and removes the entire category of risk.

14.2 Descriptions that hold up

Two versions are needed: one that is true of the repository as it stands, and one that becomes true once the rebuild is complete. Both lead with the engineering rather than with an accuracy figure.

Accurate now, before any rebuild work

Built a privacy-preserving Chrome extension that analyses images entirely on-device with no network calls, running quantized ONNX Vision Transformers through ONNX Runtime Web on a WebAssembly backend, with saliency-ranked tiling and a two-stage global-then-local pipeline offloaded to a Web Worker via zero-copy ImageBitmap transfers. Implemented ten image-forensics analyses from first principles in TypeScript, including a radix-2 Cooley-Tukey 2D FFT, Canny edge detection with non-maximum suppression and hysteresis, JPEG quantization-table estimation and PRNU sensor-noise residuals.

Every clause there is verifiable from the repository, and the forensic implementations are the strongest and least replicable part of it.

Accurate after the rebuild, and the stronger version

Distilled a large vision-foundation-model image detector into a browser-deployable student, reducing the shipped model from [teacher size] to [student size] while retaining [N] per cent of teacher AUROC, at [X] ms median on-device inference via WebGPU with a WebAssembly fallback. Published a cross-generator evaluation protocol with generator-family holdout and compression-laundering augmentation, and a temperature-calibrated abstention band tuned to a [F] per cent false-positive rate.

Every bracketed value is a measurement produced by the plan in section 11, and none of them is an accuracy claim competing against a cloud service.

14.3 Two audiences, two artifacts

A summary line receives a few seconds of attention; a technical conversation receives an hour. They reward opposite things, and the same sentence cannot serve both.

Audience	What lands	What to lead with
The skim	Concrete engineering with no caveat attached: on-device inference, compression ratio, measured latency, a published benchmark.	The pipeline and the constraint it satisfies. Not an accuracy figure — an honest number will read as worse than a competitor's unverifiable one.
The conversation	The audit, the reasoning behind calibration and abstention, and the decision not to claim a figure that cannot be defended.	The measured baseline-to-student gap, the evaluation design, and why the operating point was chosen as it was.
The accuracy number is not the strongest asset this project has and should not be asked to carry the summary. The strongest assets are the evaluation design, the compression result, and the fact that every claim is reproducible.		

14.4 How the earlier version is described

The audit findings in sections 3 and 4 are the most distinctive material available, and they belong in the written account. The framing is **version one to version two, with a measured improvement** — which is ordinary engineering practice — rather than a confession of having shipped something broken.

The distinction is not cosmetic. "I discovered the output parsing was reading the wrong tensor positions, established the correct contract empirically, and measured the improvement" describes an engineer with a working method. The same facts arranged as an apology describe someone who shipped without checking. The findings are an asset; the self-criticism is not, and the repository history is public in either case.

14.5 Claims that must not be made

- No accuracy figure without the benchmark it refers to stated alongside it.
- No comparison against competitors' published figures, which are not measured under a known protocol and are not comparable to a figure that is.
- No description of the system as detecting AI images generally. It detects them under stated conditions, at a stated operating point, and abstains elsewhere.
- No quiet substitution of a friendlier evaluation set once the first honest number is known. The entire value of the positioning depends on this not happening, and it will be tempting exactly once.

14.6 Anticipated questions

The questions below are the ones a technical interviewer is most likely to reach for. Each answer should be recoverable from the body of this document.

Question	Where the answer lives, and its shape
"Walk me through what the model actually does."	Section 10. A frozen self-supervised backbone produces an embedding; a trained head maps it to a calibrated probability; a distilled student reproduces the teacher's behaviour at deployable size. Be able to say why the backbone is frozen: it cannot overfit to the generators in the training set, which is the field's dominant failure mode.
"Why distil rather than just ship the good model?"	Section 10.2. The backbone that performs best is roughly 300 MB. A browser extension cannot ship that. Compression is not an optimisation applied after the fact; it is the constraint that determines whether the product exists.
"Why is your accuracy lower than the commercial tools?"	Sections 8 and 9.1. Their figures are not attached to a stated benchmark and are not comparable. More importantly, they are cloud services and do not attempt on-device execution, so the comparison is between different problems. Give the measured number with its benchmark named.
"How do you know the detector works at all?"	Section 11, stages 1 and 2. Generator-family holdout, laundering augmentation applied at evaluation time, AUROC with false-positive rate at 95 per cent true-positive rate, and expected calibration error. The harness was built before the model deliberately.
"What does 'inconclusive' mean, and isn't it a cop-out?"	Section 10, layer two, and the stage 5 acceptance criterion. It is a calibrated band tuned to a target false-positive rate, and the abstention rate is capped rather than free, because an unbounded band would be evasion rather than honesty.
"What was the hardest bug you found?"	Section 3. A three-class output parsed by code that handled only one and two classes, reading <code>outputData[i]</code> where the correct index was <code>3i</code> , so values were mixed across class and batch boundaries. It produced finite, plausible-looking numbers, which is why it survived. The lesson is that inference code fails silently and must assert its tensor contract at load time.
"What would you do differently?"	Build the evaluation harness before the model, and validate the model contract against a small labelled set on day one. Both are now stages 1 and 2 for that reason.
"What are the limitations?"	Sections 5 and 13. Cross-generator transfer is the field's open problem; laundering attenuates the signal; the hardest circulating imagery defeats current published detectors. State these directly rather than being led to them.

Appendix A Evidence log

Every technical claim in this document was verified directly rather than inferred. This table records how, so that any claim can be re-checked independently.

Claim	Verification method
Global model outputs logits of shape [batch, 3]; local outputs [batch, 2].	Graph inspection of both ONNX files; confirmed a second time via classifier.weight dimensions of [768, 3] and [768, 2].
Both models are ViT-Base/16.	Patch-embedding projection weights of [768, 3, 16, 16] (768 hidden dimensions, 16x16 patches) and 12 distinct encoder layers in both graphs.
The two files are distinct but near-identical in size.	Differing MD5 checksums; sizes of 87,346,871 and 87,333,629 bytes.
No label map ships with either model.	ONNX metadata_props contains only the entry onnx.infer = onnxruntime.quant. No sidecar label file exists on main.
The parsing defect and the unresolved class-index comment.	Direct reading of src/content/inference/worker.ts, lines 114 to 149 and 200 to 300.
The early-exit gate can suppress the local model entirely.	Direct reading of the gate condition and its enclosing branch in the same file.
V2 metadata states incorrect tensor names.	git show e2bb02f:public/models/model_global_meta.json, compared against the tensor names read from the model.
V2's fusion weights were never fitted; temperature is a no-op.	git show e2bb02f:public/models/fusion_v2.json.
V2 added no new model weights.	git diff --stat main..e2bb02f restricted to public/models/*.onnx returns empty.
V2 training scripts build a ResNet-50.	git show e2bb02f:scripts/train_global.py.
The offscreen document is never instantiated.	No occurrence of chrome.offscreen or createDocument anywhere under src/.
The offscreen worker references a nonexistent model.	src/offscreen/worker.ts line 27 loads models/model_quantized.onnx; only the two dual-model files exist in public/models/.
activeTab is requested but unused.	The only textual matches under src/ are an unrelated local variable in the service worker and React state in a forensic tool.
Ten forensic analyses, plus two shared UI components.	Directory listing of src/components/tools/ (12 files), with each file read to classify it.
Google's AI Pro plan does not include Colab compute.	Google's published AI plans page lists no Colab entitlement; Colab's own FAQ names no bundled subscription granting its paid tiers.

Appendix B References

Research

- A Sanity Check for AI-generated Image Detection (Chameleon benchmark)
arxiv.org/html/2406.19435v2
- Brought a Gun to a Knife Fight: Modern VFM Baselines Outgun Specialized Detectors
arxiv.org/abs/2509.12995
- Navigating the Challenges of AI-Generated Image Detection in the Wild: What Truly Matters?
arxiv.org/html/2507.10236v1
- Rethinking Cross-Generator Image Forgery Detection through DINOv3
arxiv.org/abs/2511.22471
- Community Forensics: Using Thousands of Generators to Train Fake Image Detectors (CVPR 2025)
arxiv.org/abs/2411.04125
- NTIRE 2026 Challenge on Robust AI-Generated Image Detection in the Wild
arxiv.org/abs/2604.11487
- Your AI-Generated Image Detector Can Secretly Achieve SOTA Accuracy, If Calibrated (AAAI 2026)
arxiv.org/abs/2602.01973
- Methods and Trends in Detecting AI-Generated Images: A Comprehensive Review
arxiv.org/html/2502.15176

Datasets and models

- Community Forensics dataset (CC-BY-4.0)
huggingface.co/datasets/OwensLab/CommunityForensics
- DINOv2, Apache 2.0
github.com/facebookresearch/dinov2
- DINOv3, custom Meta licence
github.com/facebookresearch/dinov3

Market and platform

- Google I/O 2026: AI-image detection in Chrome, Search and Gemini
techlicious.com/blog/google-search-chrome-gemini-detect-si-generated-images/
- C2PA adoption tracker, 2026
editorsweblog.org/2026/04/12/c2pa-adoption-tracker-platforms-content-credentials-2026
- Google AI plan contents
one.google.com/about/google-ai-plans/
- Colab FAQ and paid tiers
research.google.com/colaboratory/faq.html
- Chrome Web Store publishing requirements
developer.chrome.com/docs/webstore/publish

Sources were consulted in September 2026. Publication venues and dates are reported as stated by the sources themselves and were not independently audited; the technical findings in Appendix A were verified directly against this repository and do not depend on them.